

HW1 — Simple Streamlit Application Development

Overview

Over the next four homeworks we will build a small system that continuously tracks job postings at companies with job titles that we may care about and summarizes the core skills each posting is looking for. By HW4 it will search the web, scrape individual job pages, summarize them with an LLM, and run as a set of Docker containers.

This homework is the first step to read already-collected dataset stored in GCP Storage and build a Streamlit dashboard.

Note : Assume that your code should run on the virtual environment which includes the packages/libraries in the class git repo.

Learning Objectives

- Build a basic Streamlit app with `st.title`, `st.sidebar`, `st.checkbox`, `st.dataframe`, `st.barchart`.
- Retrieve data from GCP Storage using `google.cloud`.
- Load and reshape data with pandas: filter rows, drop duplicates, sort.
- Render a clickable link column with `st.column_config.LinkColumn()`.
- Handle dataframe's column which value is a list to get occurrences of its elements.

Background

You're given `GCP_BUCKET_NAME` and `GCP_FILE_NAME` which points at a file including job postings. Each entry looks like:

```
{
  "job_title": "Data Engineer",
  "company_dict": {
    "Anthropic": "anthropic.com/careers/jobs",
    "Google": "google.com/about/careers"
  },
  "results": [
    {
      "title": "Engineering Manager, Research Data Platform",
      "link": "https://job-boards.greenhouse.io/anthropic/jobs/5297059008",
      "qualification": {
        "Responsibilities": [
          "Work directly with researchers and the engineers supporting them to understand their workflows, identify the highest-leverage opportunities, and shape what the team builds next",
          "Own core datasets end to end: the pipelines that produce them, the schemas that define them, and the documentation and guarantees that make researchers trust them",

```

```

        "Drive convergence toward canonical datasets including the
        core data model for RL transcripts 4 that research teams standardize on",
        "Lead complex, multi-quarter projects that span several
        systems and teams, staying hands-on in the code",
        "Raise the team's technical bar through design reviews,
        mentorship, and the quality of your own work"
    ]
},
"skills": ["Platform Engineering", "Data Pipelines", "Data
Modeling", "Data Schemas", "Software Architecture", "System Design",
"Technical Strategy", "Mentorship"],
"date": "2026-08-24"
}, ...]
}

```

This is a static stand-in for what HW2 will fetch live from a real search API. The dashboard code you write here carries forward with changes/additions.

Provided Files

- `user_definition.py` — `GCP_BUCKET_NAME` and `GCP_FILE_NAME` indicating the path to the JSON file in GCP Bucket.

Files You Must Complete

`hw1.py` — implement the three pieces marked `# TODO`:

1. `retrieve_data_from_gcs(bucket_name: str, file_name: str) -> json`

Read the JSON file called `file_name` in `bucket_name` and return it as a dictionary format.

2. `summarize_distribution(df: pd.DataFrame, column_name: str, top_n: int = 10) -> dict`

Assume that `df[column_name]` holds a list per row. Inspect elements in all the list to get the `top_n` most mentioned values. For instance, if `column_name` holds 3 rows of `['python','sql','data pipeline']`, `['python','sql']`, `['python']` `top_n=2 -> {'python': 3, 'sql': 2}`

3. The sidebar filtering logic in `if __name__ == '__main__':`

1. Add title (`'job_title'` Listings and Skills) for the chosen `'job_title'` from the retrieved data.
2. Build a **sorted** list of company display names (the dict's keys) to display on the sidebar.
3. For each company, render an `st.checkbox` (default `value=True`) with the company name as its label on the side bar.
4. Filter data down to only the rows whose `"link"` column contains **any** of the substrings in the selected company in the step 2. Note: To check the substring, ensure it is case insensitive (Ex. "Anthropic" -> "https://job-boards.greenhouse.io/anthropic/jobs/5297059008") If no companies are selected, the filtered result should be empty — not "everything." (Hint: `str.contains()` accepts a regex; joining your substrings with `"|"` builds an OR pattern.) Note: If you are not familiar with regex

but want to try for this question, feel free to get GenAI's help. But ensure to attribute/cite the model you used and what you requested (ex. Anthropic opus-5, create regex pattern to filter strings containing any values in a list) as a comment

5. Keep only the "date", "title", "skills" and "link" columns.
6. Drop duplicate rows by "link".
7. Sort by "date" descending, so the newest postings show first. Optional : Sort by "title" for the same "date".
8. Display the result with `st.dataframe(...)`, using `st.column_config.LinkColumn()` on the "link" column so it renders as a clickable link rather than raw text.
9. Display the `summarize_distribution(df)` using `st.bar_chart(...)` as a bar chart.

How to Run

```
streamlit run hw1.py
```

The completed application should look like the one in [this](#)

How to Test

`test_hw1.py` is provided as a self-check (will run the identical test for grading).

```
pytest
```

This should pass all the given test cases.

Submission Checklist

- ☐ Do not add additional libraries/packages. ☐ Only extend `hw1.py`. ☐ Check to pass all `pytest` cases. ☐ `retrieve_data_from_gcs` correctly loads all 15 rows from `GCP_FILE_NAME` in `GCP_BUCKET_NAME`.
- ☐ Unchecking a company removes its postings from the table.
- ☐ Unchecking every company shows an empty table (not all rows).
- ☐ No duplicate `link` values appear in the table.
- ☐ Rows are sorted newest-first by `date` (then optionally by `title`).
- ☐ The `link` column is clickable in the rendered Streamlit app. ☐ The bar graph displays most mentioned skills (use the default `top_n` value).

What to Submit

On Canvas, submit only your `hw1.py` file.